

# Reviewer Pack — SOS Degree Lower Bound via Real Rank of Moment Matrix for Ma...

Entry 68475aeb9000 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 11:44:27 UTC

## SOS Degree Lower Bound via Real Rank of Moment Matrix for Max-CUT

Entry ID: 68475aeb9000 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 11:44:27 UTC

### 1. Conjecture

**Field A** (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

**Statement:**

For any Max-CUT instance represented as a graph  $G$ , the minimal SOS degree  $d$  required to achieve an approximation ratio of 0.878 is at least the real rank of the moment matrix derived from  $G$ 's adjacency matrix.

**Rationale (proposer's reasoning):**

The real rank of the moment matrix captures the intrinsic dimensionality of the problem's constraints, which directly influences the SOS hierarchy's necessary degree to approximate the solution. This bridge leverages the interplay between algebraic structure and optimization complexity.

**Taxonomy category:** SOS\_DEGREE (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 125459e7abae0b4b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.95       | UNCERTAIN        | SAFE             |

| Barrier       | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|---------------|---------|------------|------------------|------------------|
| ALGEBRIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON   | SAFE    | 0.95       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gram_schmidt(A):
        Q = []
        R = []
        for a in A:
            q = a[:]
            for i, q_i in enumerate(Q):
                r = sum(q_j * q_i[j] for j in range(len(a)))
                q = [q_k - r * q_i[k] for k, q_k in enumerate(q)]
            norm = math.sqrt(sum(q_k**2 for q_k in q))
            if norm == 0:
                continue
            Q.append([q_k / norm for q_k in q])
            R.append([r if i == j else 0 for j in range(len(A))])
        return Q, R

    def matrix_rank(matrix):
        Q, _ = gram_schmidt(matrix)
        rank = sum(1 for row in Q if any(row))
        return rank

    n = random.randint(40, 50)
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    A = []
    for i in range(n):
        for j in range(i + 1, n):
            if G[i][j] == 1:
                A.append([G[i][k] - G[j][k] for k in range(n)])

    real_rank = matrix_rank(A)

    def sos_relaxation(d):
        # Simulate SOS relaxation up to degree d
```

```

    # This is a placeholder function; actual implementation depends on the problem
    return False

d_min = None
for d in range(1, 11):
    if sos_relaxation(d):
        d_min = d
        break

metric_name = "SOS Degree Lower Bound"
metric_value = real_rank
instances_tested = 1
conjecture_holds = d_min is not None and d_min >= real_rank
counterexample = "" if conjecture_holds else f"Graph with n={n}, A={A}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_269dfc71.py", line 81, in <module>
    trial_result = run_trial(seed)

```

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_269dfc71.py", line 49, in run_trial
    real_rank = matrix_rank(A)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_269dfc71.py", line 37, in matrix_rank
    Q, _ = gram_schmidt(matrix)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_269dfc71.py", line 33, in gram_schmidt
    R.append([r if i == j else 0 for j in range(len(A))])
    ^
UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed due to code error before producing results; cannot evaluate conjecture validity | next: Fix the gram\_schmidt function's UnboundLocalError in the test code

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 113357       |
| 2 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 32018        |
| 3 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 27252        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20750        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 38252        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9210         |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8130         |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9522         |
| 9 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 20811        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 279301 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/68475aeb9000.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/68475aeb9000.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/68475aeb9000.tar.gz` (if generated)